Laboratório 9: n-step Sarsa (CliffWalking)

Importações

In [ ]:
```
!pip install gymnasium[toy-text]
!pip install tqdm pandas seaborn
!pip install gdown
```

```
Requirement already satisfied: gymnasium[toy-text] in /usr/local/lib/python3.
dist-packages (1.4.0)
Requirement already satisfied: numpy>=1.22.0 in /usr/local/lib/python3.13/dis
packages (from gymnasium[toy-text]) (2.4.6)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.1
dist-packages (from gymnasium[toy-text]) (3.1.2)
Requirement already satisfied: typing-extensions>=4.12.0 in /usr/local/lib/
python3.13/dist-packages (from gymnasium[toy-text]) (4.16.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/
python3.13/dist-packages (from gymnasium[toy-text]) (0.0.6)
Collecting pygame-ce>=2.1.3 (from gymnasium[toy-text])
  Downloading pygame_ce-2.5.8-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (11 kB)
Downloading pygame_ce-2.5.8-cp313-cp313-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (12.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.1/12.1 MB 64.5 MB/s  0:00:00
Installing collected packages: pygame-ce
Successfully installed pygame-ce-2.5.8
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-package
(4.67.3)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packa
(2.2.3)
Requirement already satisfied: seaborn in /usr/local/lib/python3.13/dist-pack
(0.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dis
packages (from pandas) (2.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/pytho
dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist
packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/di
packages (from pandas) (2026.6)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/
python3.13/dist-packages (from seaborn) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.68.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/d
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.3)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.13/dist-pa
(from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/
packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-pac
(from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: gdown in /usr/local/lib/python3.13/dist-packag
(5.2.2)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.13/di
packages (from gdown) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-pac
(from gdown) (4.2.7)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.13/d
packages (from gdown) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-package
(from gdown) (4.67.3)
```

```
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.13/dis
packages (from beautifulsoup4->gdown) (4.0)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/
python3.13/dist-packages (from beautifulsoup4->gdown) (4.16.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/
python3.13/dist-packages (from requests[socks]->gdown) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist
packages (from requests[socks]->gdown) (3.20)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.1
dist-packages (from requests[socks]->gdown) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.1
dist-packages (from requests[socks]->gdown) (2026.10.23)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/pytho
dist-packages (from requests[socks]->gdown) (1.7.1)
```

In [ ]:
```python
import os
import gdown
import gymnasium as gym
from gymnasium.wrappers import RecordEpisodeStatistics
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm.auto import tqdm
from typing import NamedTuple, Optional, List, Tuple
from collections import deque
```

Funções auxiliares para visualização

In [ ]:
```python
RL_UTILS_URL = "https://drive.google.com/file/d/1AEJC-YWgeZCr8pZEOOiAg6PN

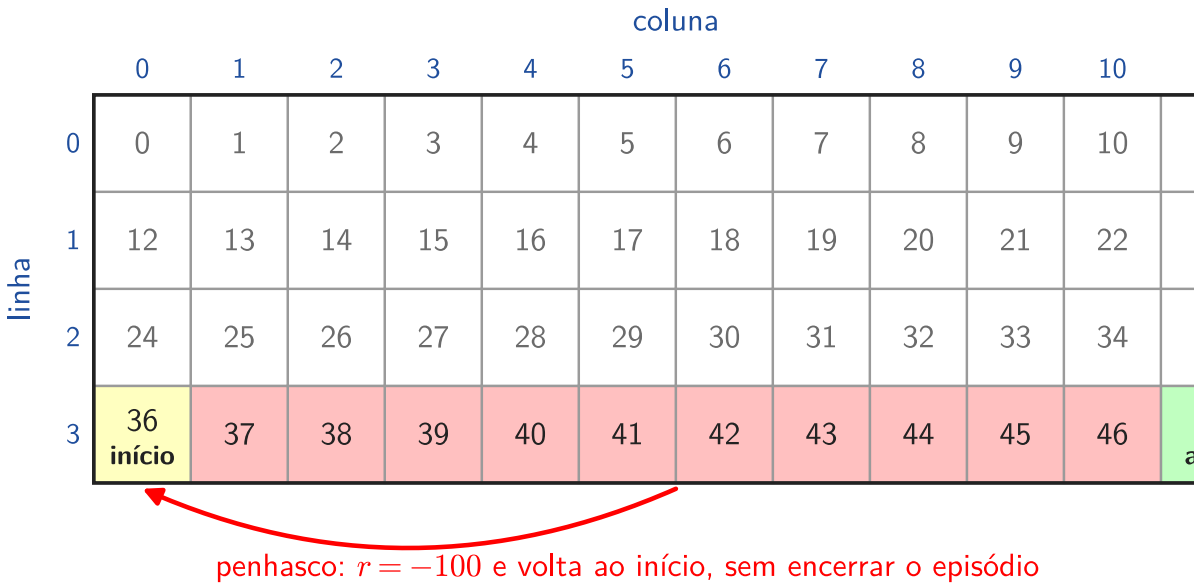
if not os.path.exists("rl_utils.py"):
    try:
        gdown.download(RL_UTILS_URL, "rl_utils.py", quiet=True, fuzzy=Tru
    except Exception as error:
        print(f"Falha no download: {error}")

from rl_utils import (
    plot_results, plot_comparison,
    simulate_trajectory, plot_trajectory,
    show_one_episode,
    plot_tabular, plot_policy_distributions,
    show_figure, show_animation,
)
```

Ambiente: nova instância

O laboratório usa o `CliffWalking-v1`, do Gymnasium.

In [ ]:
```python
show_figure("mapa_cliffwalking")
```



- **Estado:** o índice da célula, `12 * linha + coluna`, que é também a observação devol ambiente.
- **Ações:** quatro; não há a ação de permanecer no lugar.
- **Recompensa:** -1 por passo. Quanto mais curto o caminho, maior a recompensa total, e caminho mais curto passa rente ao penhasco.
- **Penhasco:** entrar nele custa -100 e devolve o agente a $s_0 = 36$, **sem** encerrar o epis
- **Fim do episódio:** há um estado terminal, o alvo. Como um agente que ainda não aprendeu vagar por muito tempo, o episódio também termina ao atingir um limite de passos.
- **Modelo:** não é usado. O agente aprende só com as amostras de experiência devolvidas po `env.step`, e não com um modelo como o `env.P` do FrozenLake (Laboratório 3).

A célula abaixo define as constantes e `make_env`, que cria o ambiente com o limite de passos envolve com o wrapper `RecordEpisodeStatistics`. Cada treinamento deve usar um ambi novo, criado por `make_env`.

Detalhes: `RecordEpisodeStatistics`

- No último passo de cada episódio, acrescenta a `info` a entrada `info["episode"]`, c
  - `"r"`: a recompensa total, isto é, a soma das recompensas, sem desconto;
  - `"l"`: a duração, em passos;
  - `"t"`: o tempo de execução, em segundos.
- Guarda esses valores dos últimos `STATS_BUFFER_LENGTH` episódios nas filas `return_queue` e `time_queue`, usadas nas médias da barra de progresso.
- Um ambiente novo começa com as filas vazias.

In [ ]:
```
ENV_ID = "CliffWalking-v1"
MAX_EPISODE_STEPS = 500
STATS_BUFFER_LENGTH = 100
SEED = 42
```

```
def make_env(render_mode: Optional[str] = "rgb_array") -> gym.Env:
    env = gym.make(ENV_ID, render_mode=render_mode,
                   max_episode_steps=MAX_EPISODE_STEPS)
    return RecordEpisodeStatistics(env, buffer_length=STATS_BUFFER_LENGTH


env = make_env()
```

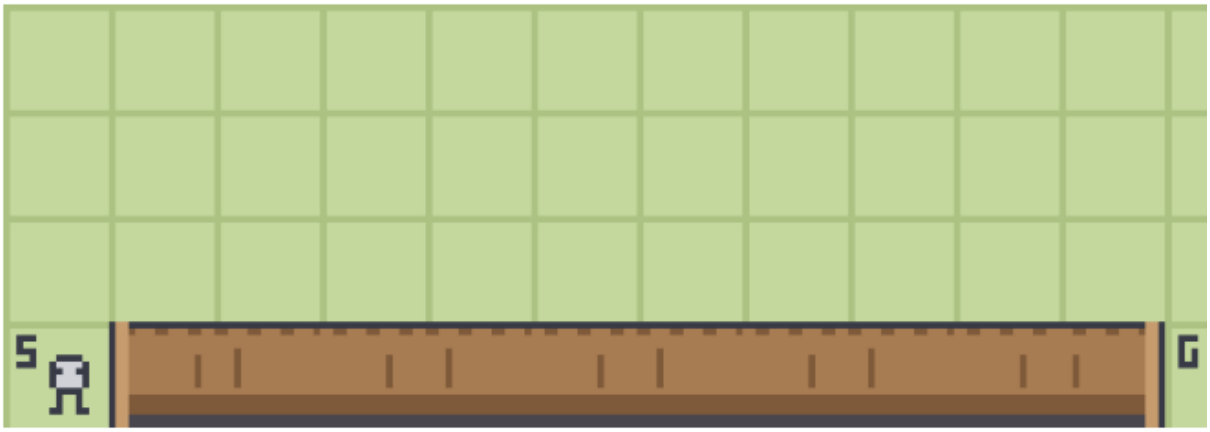
`env.reset` inicia um episódio e devolve o estado inicial s_0 e o dicionário `info`.

In [ ]:
```
obs, info = env.reset(seed=SEED)
obs, info
```

Out[ ]:
```
(36, {'prob': 1})
```

`env.render()` devolve a imagem do estado atual, um array com shape `(altura, largur`

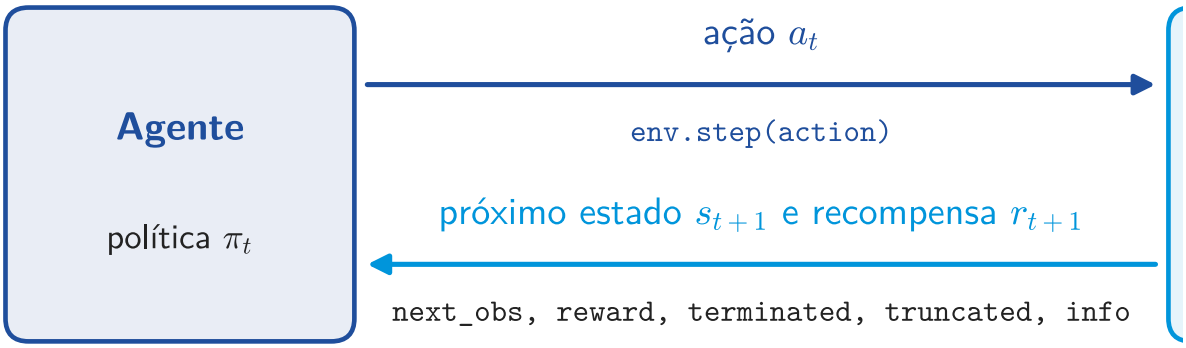
In [ ]:
```
img = env.render()
plt.figure(figsize=(9, 3))
plt.imshow(img)
plt.axis("off")
plt.show()
img.shape
```



Out[ ]:
```
(256, 768, 3)
```

`env.step` executa um passo. Ele recebe só a ação a_t, porque o ambiente já guarda o est
s_t:

In [ ]:
```
show_figure("env_step")
```



Saída	Nomenclatura	Significado
`next_obs`	s_{t+1}	próximo estado
`reward`	r_{t+1}	recompensa do passo
`terminated`	s_{t+1} terminal	`True` ao chegar ao alvo: o episódio acaba, e $v(s_{t+1})=0$
`truncated`	—	`True` ao atingir `MAX_EPISODE_STEPS`: o episódio para, mas s_{t+1} **não** é terminal
`info`	—	dados auxiliares (`"prob"` e, no último passo, `"episode"`)

> **Atenção:** a_{t+1} não vem do ambiente: o agente a sorteia em s_{t+1}, com a políti π_t.

A célula abaixo executa a ação `1` (direita) a partir de $s_0 = 36$, que leva ao penhasco. Obse `reward`, `next_obs` e `terminated`.

In [ ]:
```
next_obs, reward, terminated, truncated, info = env.step(1)
next_obs, reward, terminated, truncated, info
```

Out[ ]:
```
(36, -100, False, False, {'prob': 1.0})
```

A célula abaixo executa um episódio com ações sorteadas uniformemente por `env.action_space.sample()`. Agindo ao acaso, o agente dificilmente alcança o alvo, e o costuma acabar por truncamento; no último passo, `info["episode"]` traz as estatísticas do

In [ ]:
```
env.action_space.seed(SEED)
obs, info = env.reset(seed=SEED)
episode_over = False
while not episode_over:
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)
    episode_over = terminated or truncated
terminated, truncated, info["episode"]
```

Out[ ]:
```
(False, True, {'r': -5945.0, 'l': 500, 't': 0.019827})
```

Algoritmo: n-step Sarsa

O n-step Sarsa estende o Sarsa do Laboratório 7. No Sarsa, o alvo TD usa uma única recom r_{t+1}, e completa o resto com a estimativa $q(s_{t+1},a_{t+1})$ [*bootstrap*]. No n-step Sa alvo usa as n recompensas seguintes e só então recorre à estimativa $q(s_{t+n},a_{t+n})$ [*bo* Por isso, o par (s_t,a_t) só pode ser atualizado no instante $t+n$, quando r_{t+n}, s_{t+n} a_{t+n} já são conhecidos, e com as estimativas desse momento, q_{t+n-1}:

$$ \underbrace{q_{t+n}(s_t,a_t)}_{\substack{\text{nova}\\ \text{estimativa}}} \;=\; \underbrace{q_{t (s_t,a_t)}_{\substack{\text{estimativa}\\ \text{atual}}} \;-\; \underbrace{\alpha}_{\substack{\text{tax \text{aprendizado}}} \, \underbrace{\Big[\, q_{t+n-1}(s_t,a_t) \;-\; {\color{red}{\overbrace{\bar q_t^{ ^{\text{alvo TD}}}}} \,\Big]}_{\text{erro TD: } \delta_t^{(n)}} $$

$$ {\color{red}{\bar q_t^{(n)}} = r_{t+1} + \gamma\, r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gar

\, q_{t+n-1}(s_{t+n},a_{t+n})}} $$

- $q_t(s,a)$ é a estimativa de $q_\pi(s,a)$ no instante t.
- Cada atualização muda só um par:

$$ q_{t+n}(s,a) = q_{t+n-1}(s,a) \quad \forall(s,a) \neq (s_t,a_t) $$

- Com $n = 1$, temos o alvo do Sarsa:

$$\bar q_t^{(1)} = r_{t+1} + \gamma\, q_t(s_{t+1},a_{t+1})$$

Observações:

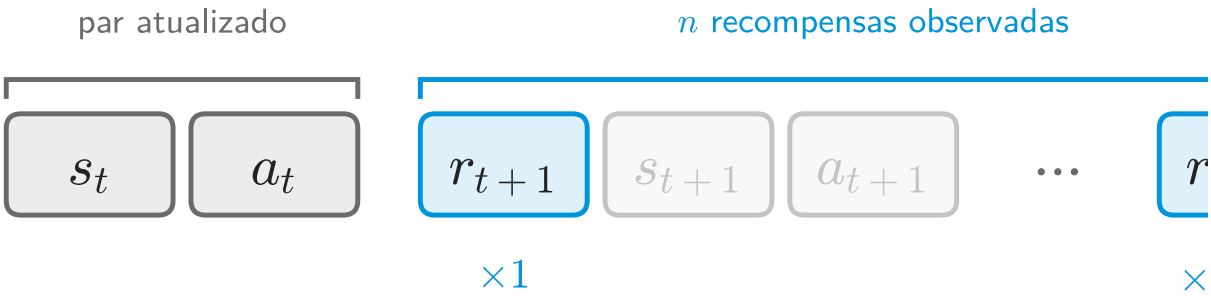
- Se o episódio **termina** no instante $T \le t + n$ (ou seja, se $n \geq T-t$) e s_T é termina
  termos faltantes são tomados como zero, pois depois do estado terminal não há recompens
  $q(s_T,\cdot) = 0$. O alvo soma as recompensas até r_T e é o retorno observado, g_t,
  mesmo do MC: $$\bar q_t^{(n)} = g_t = r_{t+1} + \gamma\, r_{t+2} + \cdots + \gamma^{T-t-1
- Se o episódio é **truncado** pelo limite de passos no instante $T \le t + n$ (ou seja, se $n \ge
  s_T não é terminal: o episódio continuaria a partir de s_T (Towers et al., 2025, Seção 4
  alvo utiliza a estimativa $q_{t+n-1}(s_T,a_T)$ [*bootstrap*]:

  $$\bar q_t^{(n)} = r_{t+1} + \gamma\, r_{t+2} + \cdots + \gamma^{T-t-1} r_T + \gamma^{T-t}\
  q_{t+n-1}(s_T,a_T)$$

O que entra no alvo, no meio e no fim do episódio:

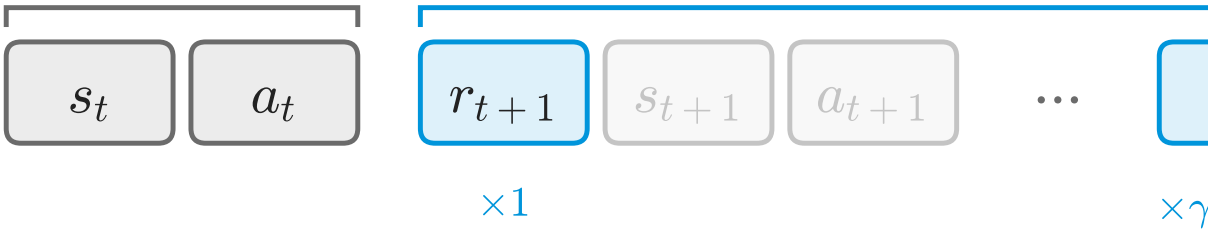
In [ ]:
```
show_figure("alvo_n_passos")
```

No meio do episódio ($t+n<T$)



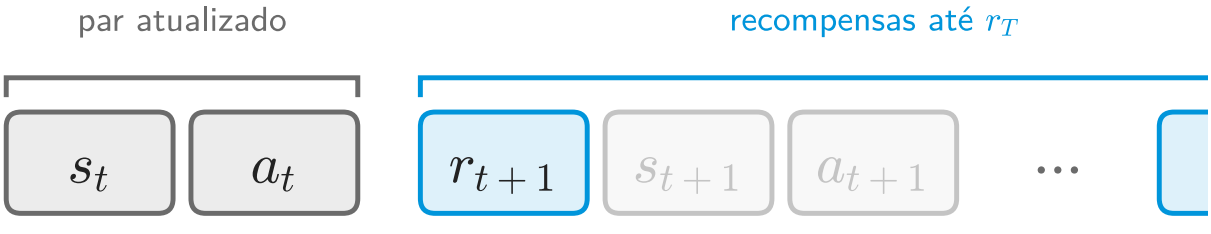
$$\bar{q}_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n}$$

Perto do fim ($t+n \geq T$), com s_T terminal



$$\bar{q}_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \cdots +$$

Perto do fim ($t+n \geq T$), com s_T não terminal (truncamento)



$$\bar{q}_t^{(n)} = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{T-t-1} r$$

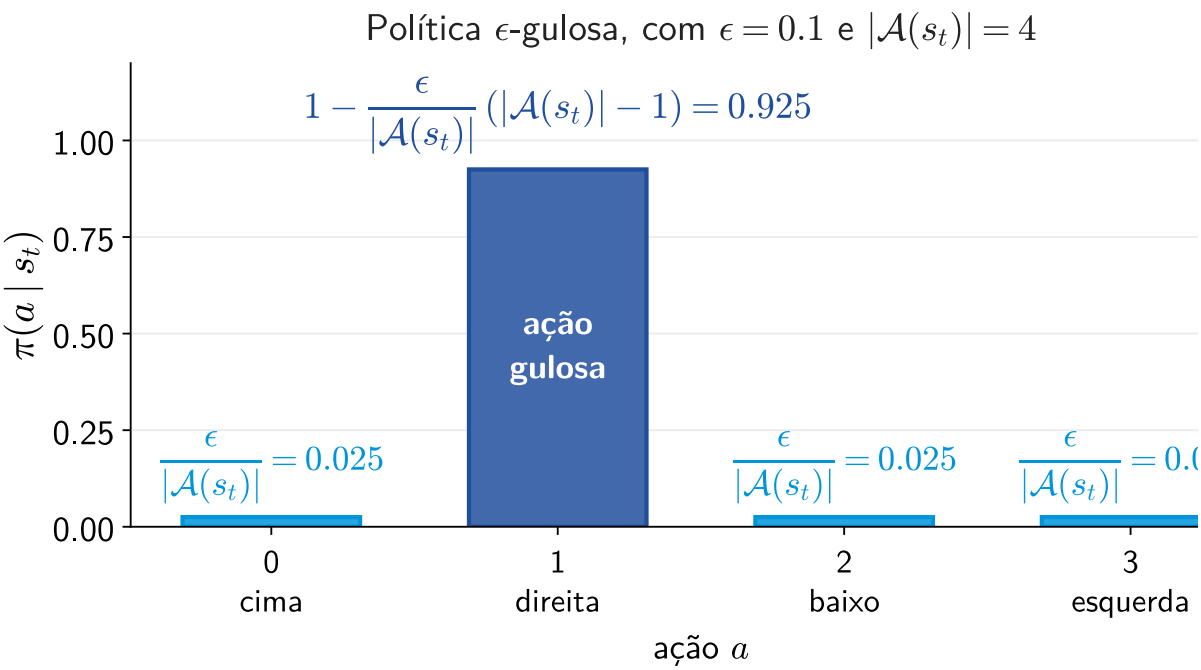
Cada atualização tem dois passos:

1. **Valor de ação:** atualiza $q(s_t, a_t)$ pela equação acima.
2. **Política:** atualiza a política em s_t, que passa a ser ϵ-gulosa em relação às esti atualizadas:

$$ \pi_{t+n}(a \mid s_t) = \begin{cases} 1 - \dfrac{\epsilon}{|\mathcal{A}(s_t)|}\big(|\mathcal{A}(s_ 1\big), & \text{se } a = \arg\max\limits_{a} q_{t+n}(s_t,a) \\[10pt] \dfrac{\epsilon}{|\mathcal{A}(s_t) \text{caso contrário} \end{cases} $$

Com $\epsilon = 0.1$ e as quatro ações do CliffWalking:

In [ ]:
```
show_figure("epsilon_gulosa")
```



> **On-policy** ($\pi_b = \pi_T$): as ações $a_{t+1}, \dots, a_{t+n}$, que definem as recompensas e a estimativa do alvo, são sorteadas pela mesma política que o agente executa. Por isso os valores aprendidos são os dessa política, exploração incluída.

Implementação. Preencha os blocos marcados:

Parte	Descrição	A implementar
`Params`	reúne os hiperparâmetros, agora com n	—
`NStepSarsaAgent`	guarda q, π e as contagens de visitas; sorteia as ações; executa os dois passos	`update_policy`, `get_target_value` e `update_Q`
`run_episode_and_train`	executa um episódio, guardando as amostras e atualizando cada par quando o alvo dele fica completo	o laço
`train_agent`	repete os episódios e registra as métricas	—

Hiperparâmetros

`Params` reúne os hiperparâmetros; `n_states` e `n_actions` vêm do ambiente.

Campo	Hiperparâmetro
`n_episodes`	número de episódios, N
`n_steps`	número de passos do alvo, n
`learning_rate`	taxa de aprendizado, α
`discount_factor`	fator de desconto, γ

Campo	Hiperparâmetro
`epsilon`	parâmetro ϵ da política

Detalhes: como variar um hiperparâmetro com `_replace`
Uma `NamedTuple` não pode ser alterada depois de criada, mas `params._replace(n_ste` devolve uma cópia de `params` com n trocado. Isso facilita variar um hiperparâmetro de cad como no item 2 da tarefa.

In [ ]:
```python
class Params(NamedTuple):
    n_states: int
    n_actions: int
    n_episodes: int
    n_steps: int
    learning_rate: float
    discount_factor: float
    epsilon: float
```

Agente

Implemente `update_policy`, `get_target_value` e `update_Q`; os demais métodos já e prontos. O agente guarda:

Atributo	Conteúdo
`self.Q`	estimativas $q_t(s,a)$, inicializada com zeros
`self.Pi`	política $\pi_t(a \mid s)$, inicializada uniforme linha `self.Pi[s]` é o vetor de probabilidad ações em `s`
`self.visit_counts`	número de visitas a cada par (s,a)
`self.n_steps`, `self.learning_rate`, `self.discount_factor`, `self.epsilon`	n, α, γ, ϵ
`self.rng`	gerador de números aleatórios do agente

As três matrizes têm shape `(n_states, n_actions)` e são indexadas por `[estado, aç` métodos de atualização recebem as amostras do episódio em três listas, com o mesmo índice e o índice τ do par a atualizar, que faz o papel de t na equação da seção **Algoritmo**:

`states[i]`	`actions[i]`	`rewards[i]`	`tau`	`T_end`	`terminated`
s_i	a_i	r_i	τ	T	s_T é terminal?

- `rewards[0]` não corresponde a nenhuma transição: vale 0 e existe só para que as três l usem o mesmo índice.
- `T_end` é o instante final do episódio, T, e vale `np.inf` enquanto o episódio não acab

Detalhes: métodos prontos

- `get_greedy_action(obs)`: devolve $\arg\max_a q_t(s,a)$; em caso de empate, a açã menor índice.

- `sample_action(obs)` : sorteia uma ação com as probabilidades de `self.Pi[obs]` , u `self.rng` , e devolve `(action, is_exploratory)` , em que `is_exploratory` é ação sorteada difere da gulosa e `0` caso contrário.
- `update(states, actions, rewards, tau, T_end, terminated)` : chama `upd` e `update_policy` em s_τ e registra a visita ao par (s_τ, a_τ).
- `policy(obs, deterministic)` : devolve a ação gulosa ou sorteia uma ação $\epsilon gulosa; é usado nas visualizações.

A implementar:

- `update_policy(obs)` $\to$ não devolve nada; atualiza a linha `self.Pi[obs]` , o pas pela fórmula ϵ-gulosa da seção **Algoritmo**:
  - a ação gulosa recebe $1 - \dfrac{\epsilon}{|\mathcal{A}(s_t)|}\big(|\mathcal{A}(s_t)| - 1\b
  - cada uma das demais ações recebe $\dfrac{\epsilon}{|\mathcal{A}(s_t)|}$, com $|\math (s_t)|$ = `self.n_actions` .
- `get_target_value(states, actions, rewards, tau, T_end, terminated)` alvo $\bar q_\tau^{(n)}$, um número real:
  - τ é o instante do par (s_τ, a_τ) cuja estimativa está sendo atualizada (o $t seção **Algoritmo**); no laço, $\tau = t - n + 1$, em que t é o instante corrente;
  - $h = \min(\tau + n, T)$ é o índice da última recompensa do alvo: `h = min(tau + self.n_steps, T_end)` ;
  - soma as recompensas: $r_{\tau+1} + \gamma\, r_{\tau+2} + \cdots + \gamma^{h-\tau-1}
  - acrescenta a estimativa $\gamma^{h-\tau}\, q(s_h,a_h)$, exceto se s_h é terminal, is $h = T$ e `terminated` é `True` .
- `update_Q(states, actions, rewards, tau, T_end, terminated)` $\to$ o err \delta_\tau^{(n)}$, um número real (escalar):
  - calcula o alvo com `get_target_value` e o erro TD: $\delta_\tau^{(n)} = q(s_\tau,a_ \bar q_\tau^{(n)}$;
  - atualiza o par (s_τ, a_τ): $q_{\tau+n}(s_\tau,a_\tau) = q_{\tau+n-1}(s_\tau,a_\ta \alpha\,\delta_\tau^{(n)}$.

> **Atenção:**
>
> - Só `terminated` dispensa a estimativa. No truncamento, s_T não é terminal, e \gamma^{T-\tau}\, q(s_T,a_T)$ continua no alvo; por isso os métodos não recebem `truncated` .
> - Com `T_end = np.inf` , `min(tau + self.n_steps, T_end)` devolve o inteiro `tau + self.n_steps` .
> - Em `update_policy` , obtenha a ação gulosa com `self.get_greedy_action(obs)` : é a mesma que `sample_action` usa para marcar as ações exploratórias.

In [ ]:
```python
class NStepSarsaAgent:
    def __init__(self, params: Params, seed: int = SEED):
        # Número total de estados no ambiente (48 estados)
        self.n_states = params.n_states
        # Número de ações possíveis (4 ações)
```

```
        self.n_actions = params.n_actions
        # Número de passos n do alvo multi-passos (ex: n=4)
        self.n_steps = params.n_steps
        # Taxa de aprendizado alpha (ex: 0.01)
        self.learning_rate = params.learning_rate
        # Fator de desconto temporal gamma (ex: 0.9)
        self.discount_factor = params.discount_factor
        # Epsilon para exploração comportamental (ex: 0.1)
        self.epsilon = params.epsilon
        # Gerador pseudoaleatório com semente fixa
        self.rng = np.random.default_rng(seed)
        shape = (self.n_states, self.n_actions)
        # Tabela Q: estimativas q(s, a)
        self.Q = np.zeros(shape)
        # Política Pi: inicializada uniforme (0.25 para cada ação)
        self.Pi = np.full(shape, 1.0 / self.n_actions)
        # Contador de visitas a cada par (estado, ação)
        self.visit_counts = np.zeros(shape)


    def get_greedy_action(self, obs: int) -> int:
        # Retorna o índice da ação com maior valor Q no estado obs
        # Em empate, retorna a primeira ação (índice 0 = CIMA)
        return int(np.argmax(self.Q[obs]))


    def update_policy(self, obs: int) -> None:
        # Identifica a melhor ação conhecida no estado obs
        greedy_action = self.get_greedy_action(obs)
        # Probabilidade base de exploração (epsilon / 4)
        base = self.epsilon / self.n_actions
        # Probabilidade da melhor ação: (1 - epsilon) + fatia base
        calc = (1.0 - self.epsilon) + base
        self.Pi[obs] = base
        self.Pi[obs, greedy_action] = calc


    def sample_action(self, obs: int) -> Tuple[int, int]:
        # Sorteia ação de acordo com as probabilidades em Pi[obs]
        action = int(self.rng.choice(self.n_actions, p=self.Pi[obs]))
        # Marca se a ação foi exploratória (diferente da melhor ação)
        is_exploratory = int(action != self.get_greedy_action(obs))
        return action, is_exploratory


    def policy(self, obs: int, deterministic: bool = False) -> int:
        # Modo guloso puro para deploy/avaliação sem exploração
        if deterministic:
            return self.get_greedy_action(obs)
        # Modo estocástico epsilon-guloso para treino
        return self.sample_action(obs)[0]


    def get_target_value(self, states: List[int], actions: List[int],
                         rewards: List[float], tau: int, T_end: float,
                         terminated: bool) -> float:
        # 1. Define o horizonte h: avança n passos a partir de tau, sem u
        h = int(min(tau + self.n_steps, T_end))
        target_value = 0.0
```

```
        # 2. Acumula a soma descontada das recompensas reais observadas e
        # target += gamma^(i - tau - 1) * R_i
        for i in range(tau + 1, h + 1):
            target_value += (self.discount_factor ** (i - tau - 1)) * rew

        # 3. Bootstrap: Se o horizonte h NÃO for o término do episódio, a
        # Se atingiu o final do episódio terminal (h == T_end e terminate
        if not (h == T_end and terminated):
            target_value += (self.discount_factor ** (h - tau)) * self.Q[

        return target_value

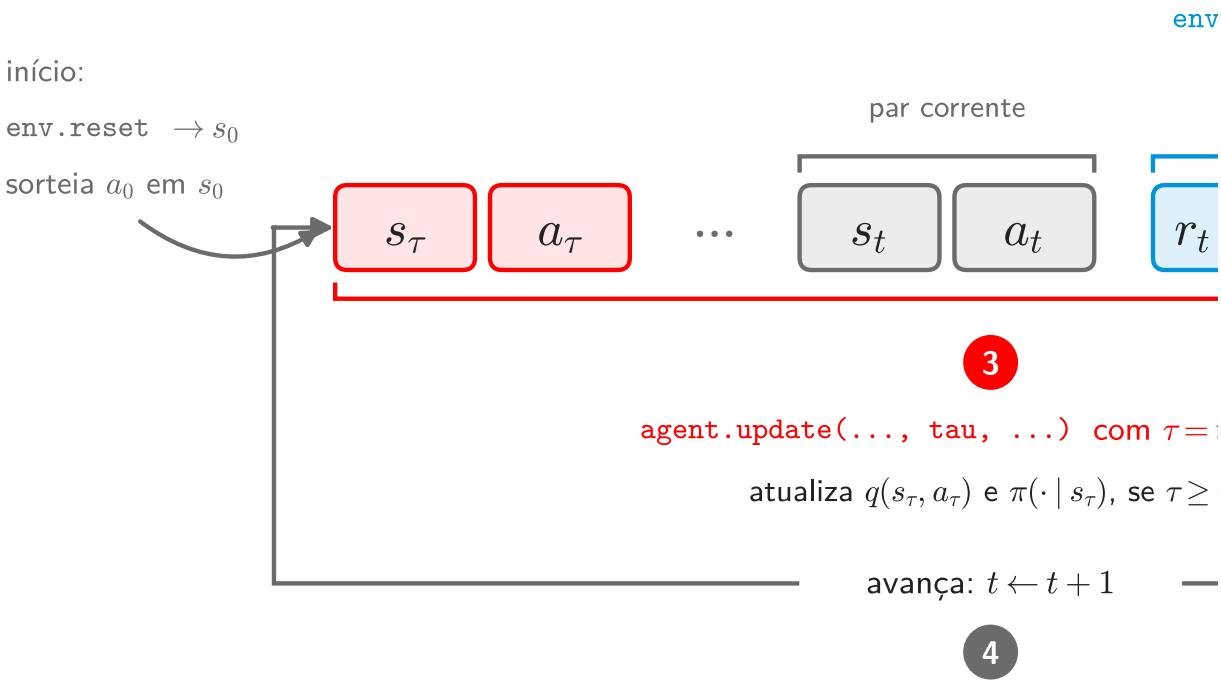

    def update_Q(self, states: List[int], actions: List[int],
                 rewards: List[float], tau: int, T_end: float,
                 terminated: bool) -> float:
        # 1. Calcula o alvo n-step para o par (s_tau, a_tau)
        target_value = self.get_target_value(states, actions, rewards, ta
        # 2. Calcula o erro TD n-step: Q(s_tau, a_tau) - G_{tau:tau+n}
        td_error = self.Q[states[tau], actions[tau]] - target_value
        # 3. Atualiza a tabela Q na posição defasada (s_tau, a_tau)
        self.Q[states[tau], actions[tau]] -= self.learning_rate * td_erro
        return td_error


    def update(self, states: List[int], actions: List[int],
               rewards: List[float], tau: int, T_end: float,
               terminated: bool) -> None:
        # 1. Atualiza Q no estado defasado s_tau
        self.update_Q(states, actions, rewards, tau, T_end, terminated)
        # 2. Atualiza a política no estado s_tau
        self.update_policy(states[tau])
        # 3. Incrementa visitas
        self.visit_counts[states[tau], actions[tau]] += 1
```

Episódio de treinamento

Implemente o laço de `run_episode_and_train`. O início já está pronto: `env.reset` devo s_0, a política sorteia a_0, e as listas `states`, `actions` e `rewards` começam com $ a_0 e $r_0 = 0$. Em cada iteração, o instante corrente é t, e o par cujo alvo acaba de ficar é (s_τ, a_τ), com $\tau = t - n + 1$. A iteração segue os passos da figura:

In [ ]:
```
show_figure("nstep_passo")
```



Se $t < T$:

1. executa a_t com `env.step` e guarda r_{t+1} e s_{t+1}; se esse passo encerrou o registra $T = t + 1$ em `T_end`;
2. sorteia a_{t+1} em s_{t+1} e a guarda.

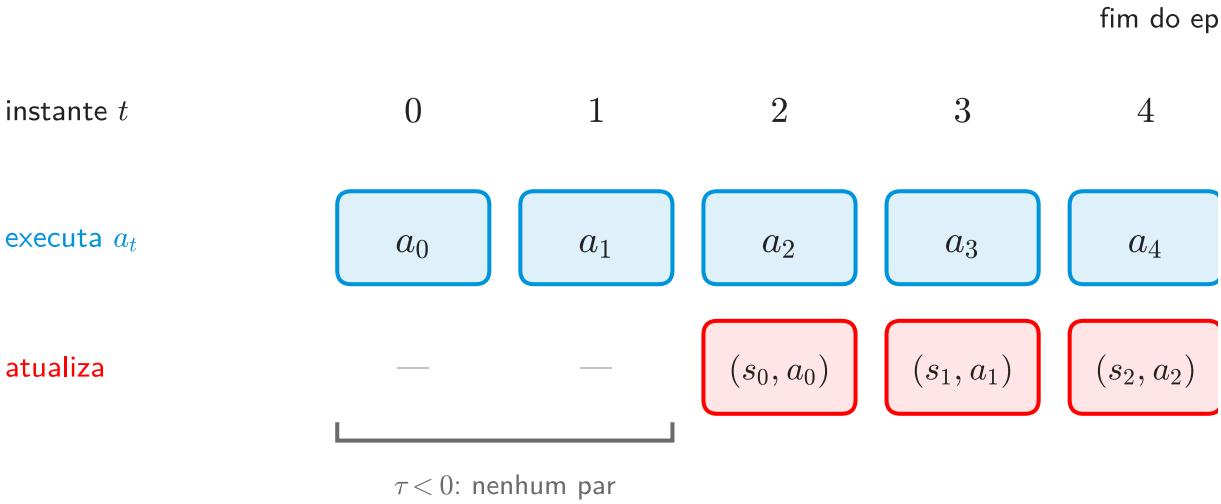
Em seguida, em todas as iterações:

1. calcula $\tau = t - n + 1$ e, se $\tau \geq 0$, atualiza o par (s_τ, a_τ), chamando `agent.update`;
2. avança: $t \leftarrow t + 1$.

O laço termina depois da atualização do último par, $\tau = T - 1$. Num episódio curto:

In [ ]:
```
show_figure("nstep_atraso")
```

Exemplo: $n = 3$ e um episódio de $T = 5$ passos



A tabela detalha o mesmo exemplo, com $n = 3$ e $T = 5$. Cada linha é uma iteração do laço: colunas r_{t+1}, s_{t+1} e a_{t+1} mostram o que ela guarda, e as demais, a atualização \tau = t - n + 1$, $h = \min(\tau + n, T)$ e o alvo

$$\bar q_\tau^{(n)} = r_{\tau+1} + \gamma\, r_{\tau+2} + \cdots + \gamma^{h-\tau-1} r_h + \gamr \tau}\, q_{\tau+n-1}(s_h,a_h),$$

em que a última parcela contendo a estimativa sai quando $h = T$ e s_T é terminal.

t	T	$ r_{t+1} $	$ s_{t+1} $	$ a_{t+1} $	$ \tau$	par atualizado	h	estimativa	alvo $\ $ .....
0	∞	r_1	s_1	a_1	-2	—	—	—	—
1	∞	r_2	s_2	a_2	-1	—	—	—	—
2	∞	r_3	s_3	a_3	0	(s_0, a_0)	3	sim	$r_1 + \ \gamma q_2(s_
3	∞	r_4	s_4	a_4	1	(s_1, a_1)	4	sim	$r_2 + \ \gamma q_3(s_
4	$T \leftarrow 5$	r_5	s_5	a_5	2	(s_2, a_2)	5	não	$r_3 + \ \gamma
5	5	—	—	—	3	(s_3, a_3)	5	não	$r_4 + \
6	5	—	—	—	4	(s_4, a_4)	5	não	r_5

- No início, $T = \infty$: o episódio ainda não terminou.
- Em $t = 0$ e $t = 1$, $\tau < 0$: nenhum alvo está completo, e nenhum par é atualizado.
- Em $t = 4$, o `env.step` devolve s_5, terminal, e o laço registra $T = 5$ **antes** de atua (s_2, a_2)$; por isso esse alvo já sai sem estimativa. A ação a_5 é sorteada, mas não é
- A estimativa só entra nos alvos com $\tau + n < T$ (ou seja, $n<T-\tau$), nas linhas $t = 2$ 3$; nas linhas $t = 4$, 5 e 6, $h = T$ e s_5 é terminal.
- Em $t = 5$ e $t = 6$, o episódio já acabou: o laço não chama `env.step` e só atualiza os que faltam.
- O laço para depois de atualizar o par de $\tau = T - 1 = 4$, em $t = 6$.

> **Atenção:**
>
> - Sorteie a_{t+1} logo depois de cada `env.step`, inclusive no último: no truncamento, a_T entra no alvo dos últimos pares, mesmo sem ser executada.
> - Depois do fim do episódio, o laço continua sem chamar `env.step`, até atualizar o pares que faltam.
> - Passe `terminated` ao agente, e **não** `terminated or truncated`.
> - Some em `n_exploratory_actions` o `is_exploratory` de cada ação **executada**; a_T não é executada.

Retorno (já pronto):

- `info["episode"]`: recompensa total, duração e tempo do episódio;
- `n_exploratory_actions`: número de ações não gulosas executadas;
- `terminated`: o sinal do último passo.

In [ ]:

```python
def run_episode_and_train(agent: NStepSarsaAgent, env: gym.Env, seed: int
    # Reseta o ambiente para o estado inicial s_0 (36)
    obs, _info = env.reset(seed=seed)
    # Sorteia a primeira ação a_0
    action, is_exploratory = agent.sample_action(obs)

    # Buffers (listas) de memória para armazenar o histórico do episódio:
    # states armazena [s_0, s_1, ...], actions armazena [a_0, a_1, ...]
    # rewards armazena [0, R_1, R_2, ...] (o índice 0 é fictício pois não
    states, actions, rewards = [obs], [action], [0.0]
    is_exploratories = [is_exploratory]

    # T_end é o passo final do episódio (inicializado como infinito enqua
    T_end = np.inf
    n_exploratory_actions = 0
    terminated = False
    info = {}
    # t: passo de tempo atual do ambiente; tau: índice do estado passado
    t, tau = 0, -1

    # O laço DEVE rodar até que tau atinja T_end - 1 para esgotar as atua
    while tau < T_end - 1:
        # Enquanto o ambiente ainda estiver ativo (antes de terminar):
        if t < T_end:
            # 1. Executa a ação do passo atual actions[t]
            next_obs, reward, terminated_step, truncated_step, info_step
            # Armazena a transição nos buffers
            states.append(next_obs)
            rewards.append(reward)
            info = info_step
            n_exploratory_actions += is_exploratories[t]

            # Se o episódio terminou naturalmente ou por limite de 500 pa
            if terminated_step or truncated_step:
                T_end = t + 1  # Fixa o tempo final do episódio
                terminated = terminated_step

            # Sorteia a próxima ação a_{t+1} e guarda no buffer de ações
            next_action, next_is_exploratory = agent.sample_action(next_o
            actions.append(next_action)
            is_exploratories.append(next_is_exploratory)

        # 2. CALCULA O ÍNDICE DEFASADO tau: tau = t - n + 1
        tau = t - agent.n_steps + 1

        # 3. Se tau >= 0, significa que já acumulamos n passos no buffer
        if tau >= 0:
            agent.update(states, actions, rewards, tau, T_end, terminated

        # Avança o contador de tempo
        t += 1
```

```
    return info["episode"], n_exploratory_actions, terminated
```

Laço externo

`train_agent` já está pronta. Ela repete `run_episode_and_train` por `n_episodes` ep
cada um com uma semente sorteada pelo gerador do agente, e devolve as séries numa
`TrainingHistory`:

Campo	Série, por episódio
`returns`	recompensa total
`lengths`	duração, em passos
`times`	tempo de execução, em segundos
`exploration_rates`	proporção de ações não gulosas executadas
`successes`	1 se o episódio terminou no alvo; como o alvo é o único estado terminal do CliffWalking, o sucesso é o próprio `terminated`

A barra de progresso mostra as médias dos últimos `STATS_BUFFER_LENGTH` episódios, calcu
com as filas do `RecordEpisodeStatistics`.

Detalhes: a tupla nomeada `TrainingHistory`
Cada campo pode ser acessado pelo nome, como em `history.returns`, e a tupla também
desempacotada na ordem em que os campos foram definidos.

In [ ]:
```
class TrainingHistory(NamedTuple):
    returns: List[float]
    lengths: List[int]
    times: List[float]
    exploration_rates: List[float]
    successes: List[int]


def train_agent(agent: NStepSarsaAgent, env: gym.Env,
                n_episodes: int) -> TrainingHistory:
    returns, lengths, times, exploration_rates, successes = [], [], [], [
    success_queue = deque(maxlen=env.return_queue.maxlen)
    pbar = tqdm(range(n_episodes), desc="Treinando", unit="episódio")

    for episode in pbar:
        seed = int(agent.rng.integers(2**32))
        episode_data, n_exploratory_actions, terminated = run_episode_and
            agent=agent, env=env, seed=seed)

        episode_length = int(episode_data["l"])
        exploration_rate = n_exploratory_actions / episode_length
        success = int(terminated)

        returns.append(float(episode_data["r"]))
        lengths.append(episode_length)
        times.append(float(episode_data["t"]))
        exploration_rates.append(exploration_rate)
        successes.append(success)
```

```
        success_queue.append(success)

        pbar.set_postfix({
            "recompensa_media": f"{np.mean(env.return_queue):.1f}",
            "passos_medio": f"{np.mean(env.length_queue):.1f}",
            "tempo_medio": f"{np.mean(env.time_queue):.3f}",
            "exploracao": f"{exploration_rate:.3f}",
            "sucesso": f"{np.mean(success_queue):.3f}",
        })

    return TrainingHistory(returns, lengths, times, exploration_rates, su
```

Experimento

O laço em câmera lenta

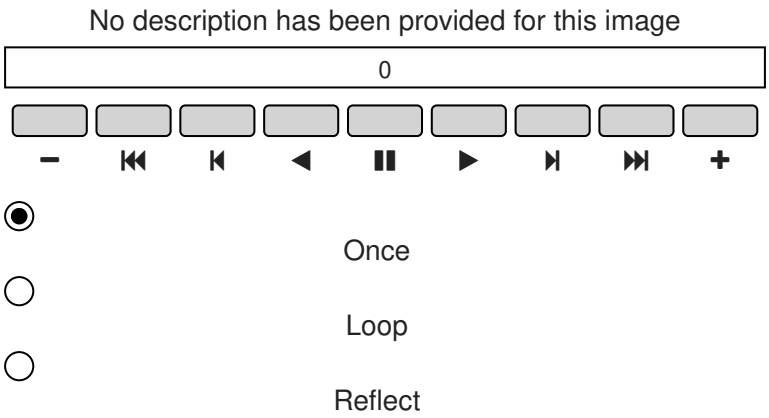
A animação abaixo mostra o laço de `run_episode_and_train` em câmera lenta, com $n = = 4$, um painel acima do outro, num episódio roteirizado de 23 passos: as ações vêm de uma li em vez de sorteadas. No oitavo passo, o agente cai no penhasco, recebe -100 e volta ao iníci encerrar o episódio; depois, segue até o alvo. Os painéis andam juntos: cada iteração t do laç dois quadros, e os dois agentes estão sempre na mesma célula:

- **`env.step`:** o agente executa a_t e chega a s_{t+1}, cuja célula ganha contorno azul aparece tracejada em vermelho;
- **atualização:** o par (s_τ, a_τ), em vermelho, recebe o alvo, e as células em azul são estados alcançados pelas transições cujas recompensas entram nesse alvo. Enquanto $\ta nenhum alvo está completo, e o painel de $n = 4$ repete o quadro.

Depois do último `env.step`, o painel de $n = 4$ ainda atualiza os pares que faltam, e o de $n cujo laço já terminou, fica parado. O canto inferior esquerdo de cada célula visitada mostra o est s_t, com um índice por visita. Cada par atualizado ganha uma seta com o valor de q: em ve o par do quadro; em cinza, os anteriores. Embaixo de cada painel, o quadro de cálculo mostra $ n, τ e h, o alvo $\bar q_\tau^{(n)}$ na forma geral, com os índices do quadro e com os números, e a atualização, com $\alpha = 0.1$ e $\gamma = 0.9$.

A animação já vem pronta: foi gerada com uma implementação correta do `NStepSarsaAgent` usa o seu código.

In [ ]:
```
show_animation("laco_camera_lenta")
```



Simulação

A célula abaixo treina o n-step Sarsa na configuração base, com $n = 4$, um ambiente (`train_env`) e um agente novos. O gerador do agente controla todos os sorteios e o CliffWall determinístico: com a mesma semente, a execução é reprodutível.

In [ ]:
```
params = Params(
    n_states=env.observation_space.n,
    n_actions=env.action_space.n,
    n_episodes=5000,
    n_steps=4,
    learning_rate=0.01,
    discount_factor=0.9,
    epsilon=0.1,
)

train_env = make_env()
agent = NStepSarsaAgent(params, seed=SEED)
history = train_agent(agent, train_env, params.n_episodes)
```

Os valores de estado da política aprendida saem das estimativas e da política finais: a média dc de ação, ponderada pelas probabilidades da política ϵ-gulosa,

$$v(s) = \sum_a \pi(a \mid s)\, q(s,a)$$

In [ ]:
```
V = np.sum(agent.Pi * agent.Q, axis=1)
```

Avaliação da política aprendida

A saída do n-step Sarsa é a política ϵ-gulosa aprendida, guardada em `agent.Pi`. `evaluate_policy`, já pronta, mede o desempenho dessa política: executa `n_episodes` e (100, por padrão) num ambiente novo, criado por `make_env`, sorteando cada ação com as probabilidades de `agent.Pi`, **sem** atualizar q nem π. Ela devolve:

Métrica	Significado
`chegada ao alvo`	proporção dos episódios que terminam no alvo
`recompensa total média`	média da recompensa total dos episódios
`duração média`	média da duração dos episódios, em passos

Os sorteios usam um gerador próprio, criado com `seed`.

> **Atenção:** as curvas de `plot_results` vêm do treinamento, em que a política muda a cada atualização, inclusive dentro de um episódio. Elas mostram como o agente aprendeu; o desempenho da política aprendida é o que `evaluate_policy` mede, co q e π fixos.

In [ ]:
```
def evaluate_policy(agent: NStepSarsaAgent, n_episodes: int = 1000,
                    seed: int = SEED) -> pd.Series:
    rng = np.random.default_rng(seed)
```

```
    eval_env = make_env(render_mode=None)
    returns, lengths, successes = [], [], []

    for _ in range(n_episodes):
        obs, _info = eval_env.reset(seed=int(rng.integers(2**32)))
        terminated = truncated = False
        while not (terminated or truncated):
            action = int(rng.choice(agent.n_actions, p=agent.Pi[obs]))
            obs, _reward, terminated, truncated, info = eval_env.step(act
        returns.append(float(info["episode"]["r"]))
        lengths.append(int(info["episode"]["l"]))
        successes.append(int(terminated))

    return pd.Series({
        "chegada ao alvo": np.mean(successes),
        "recompensa total média": np.mean(returns),
        "duração média": np.mean(lengths),
    })
```

Na configuração base:

In [ ]:
```
evaluate_policy(agent)
```

Out[ ]:

	0
chegada ao alvo	1.000
recompensa total média	-21.677
duração média	19.202

dtype: float64

Visualização

As figuras vêm das funções do `rl_utils`, importadas no início do notebook:

Função	Descrição
`plot_results`	recompensa total, duração, taxa de exploração e sucesso por ep com a média móvel de `STATS_BUFFER_LENGTH` episódios
`simulate_trajectory`	executa um episódio num ambiente próprio e devolve a trajetória padrão, a trajetória gulosa (definida abaixo)
`plot_trajectory`	o caminho sobre a grade, com o penhasco em vermelho e o alv verde
`show_one_episode`	a animação de um episódio com a política aprendida (ϵ gulosa), exploração incluída
`plot_tabular`	mapas de calor de $q(s,a)$, $\pi(a \mid s)$ ou $v(s)$, escolhido argumento `kind`
`plot_policy_distributions`	as probabilidades das quatro ações em cada estado
`plot_comparison`	as médias móveis de várias execuções num só gráfico, com lege (usada na tarefa)

> **Atenção:** cada mapa de calor calcula a escala de cores a partir dos próprios valores, e a mesma cor pode representar números diferentes em figuras diferentes. Para comparar figuras, use os números anotados nas células.

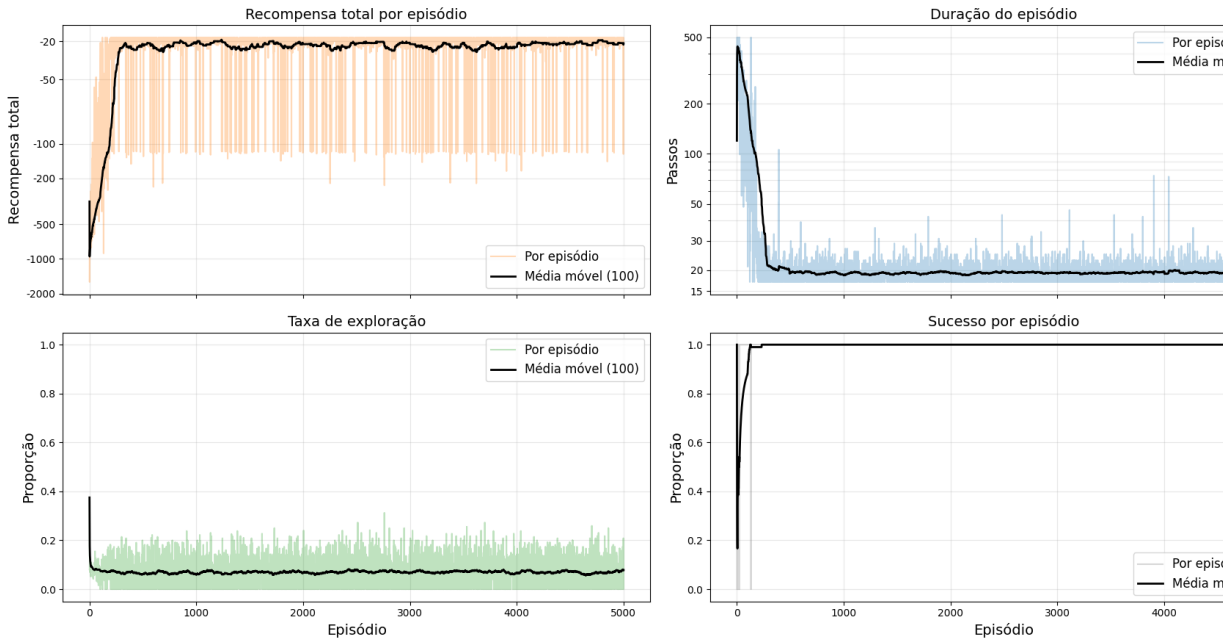
Detalhes: escalas dos eixos e gráficos lado a lado

- Em `plot_results` e `plot_comparison`, o eixo da recompensa total usa a escala `sy` quando há valores abaixo de -100: linear entre -100 e 100 e logarítmica fora dessa f eixo da duração usa a escala logarítmica quando a duração varia muito. Nos demais casos escala é linear.

- `plot_comparison` e `plot_trajectory` aceitam o argumento `ax`, o painel da figura o gráfico é desenhado. Com `ax`, elas não chamam `plt.show()`, e a figura é exibida un só, ao final. Para as curvas de duração e de recompensa total lado a lado:

  ```
  fig, axs = plt.subplots(1, 2, figsize=(18, 5))
  plot_comparison(histories, metric="lengths", ax=axs[0])
  plot_comparison(histories, metric="returns", ax=axs[1])
  plt.tight_layout()
  plt.show()
  ```
  em que `histories` é um dicionário que associa o rótulo de cada configuração (o texto da legenda) ao histórico devolvido por `train_agent`.

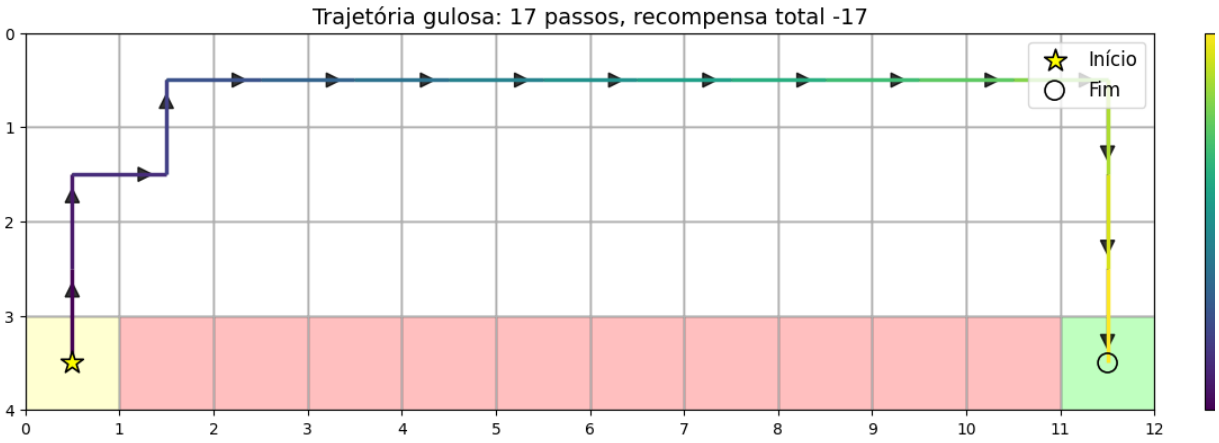
In [ ]:
```
df_results = plot_results(
    history.returns, history.lengths, history.exploration_rates,
    history.successes, window=STATS_BUFFER_LENGTH)
```



Trajetória gulosa: a trajetória obtida executando, em cada estado, a ação mais provável da polí aprendida, $\arg\max_a q(s,a)$. Ela ajuda a ler o que o agente aprendeu, mas não é a saída do step Sarsa: a política aprendida é a ϵ-gulosa, avaliada por `evaluate_policy`. Com \epsilon = 0.1$, a ação gulosa tem probabilidade 0.925 em cada estado.

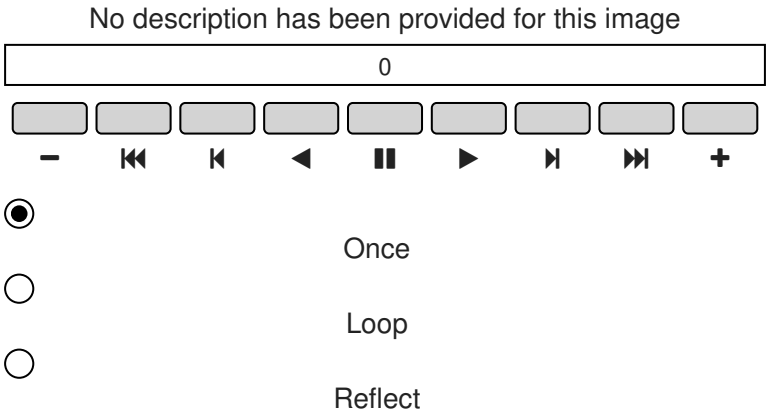
In [ ]:
```
```

```
states, actions, rewards = simulate_trajectory(agent, ENV_ID)
title = (f"Trajetória gulosa: {len(actions)} passos, "
         f"recompensa total {sum(rewards):.0f}")
_ = plot_trajectory(ENV_ID, states, title=title)
```



In [ ]:
```
show_one_episode(agent.policy, ENV_ID, seed=SEED)
```

Out[ ]:



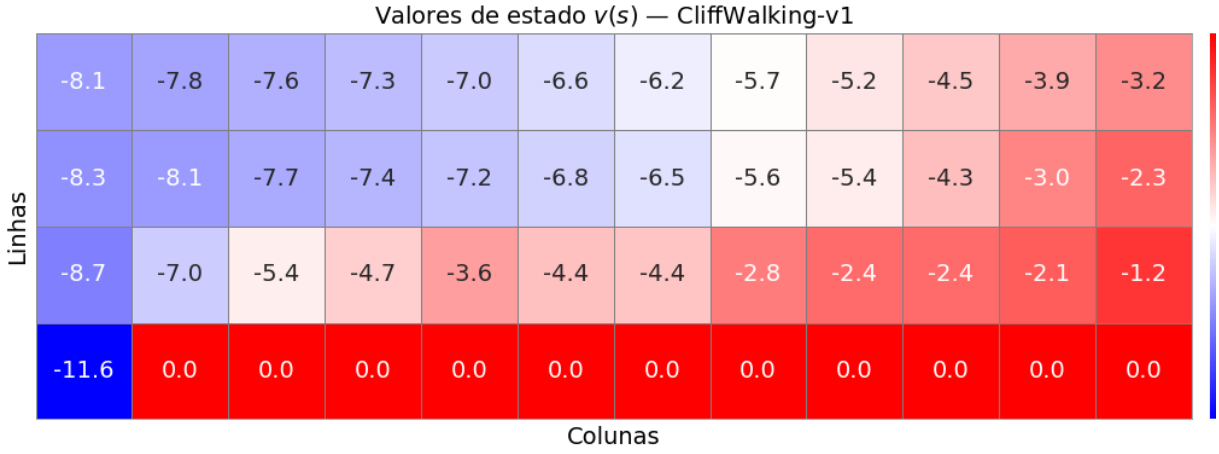
In [ ]:
```
_ = plot_tabular(agent.Q, kind="Q")
```



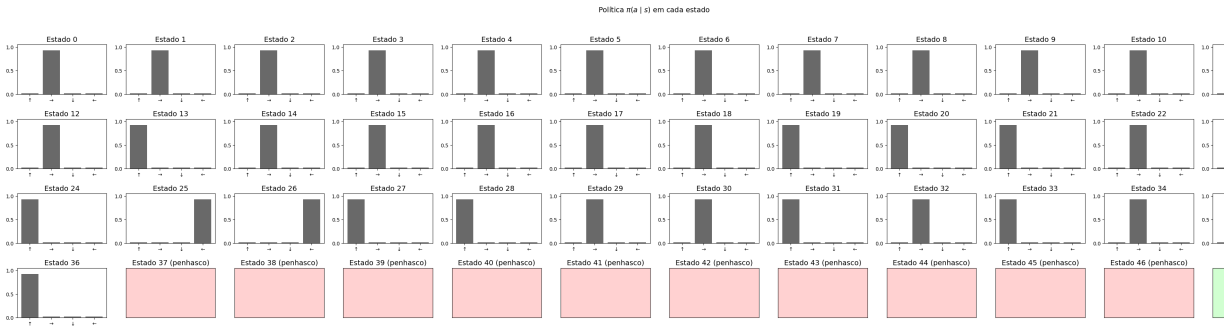
In [ ]:
```
_ = plot_tabular(agent.Pi, kind="Pi")
```



In [ ]:
```
_ = plot_tabular(V, kind="V", env_id=ENV_ID, center_zero=False)
```



In [ ]:
```
plot_policy_distributions(agent.Pi, ENV_ID)
```



Tarefa (em trios):

Responda no próprio notebook. Cada item tem uma seção **Resposta**, com:

- uma célula de código, para os experimentos, as figuras e as tabelas;
- uma célula de texto, com **uma resposta por pergunta**, identificada pelo mesmo rótulo: (a),

Acrescente células se precisar, mas não apague nem reordene os enunciados.

Configuração base (baseline)

Parâmetro	Valor
ambiente	`CliffWalking-v1`, com `MAX_EPISODE_STEPS = 500`
número de passos do alvo, n	`n_steps = 4`
número de episódios, N	`n_episodes = 5000`
taxa de aprendizado, α	`learning_rate = 0.01`
fator de desconto, γ	`discount_factor = 0.9`
parâmetro ϵ	`epsilon = 0.1`
janela das médias móveis	`STATS_BUFFER_LENGTH = 100` episódios
semente	`SEED = 42`, com ambiente (`make_env`) e agente novos em cada execução

Mostrar

- **Figuras:**
  - configuração base: as figuras da seção **Visualização** (curvas de `plot_results`, tra
    gulosa e mapas de calor de $q(s,a)$, $\pi(a \mid s)$ e $v(s)$);
  - item 1: as curvas de duração e de recompensa total de todas as execuções, sobrepost
    legenda (`plot_comparison`), e a trajetória gulosa de cada execução.
- **Saídas e tabelas:** a tabela do item 2, como pedido no enunciado.
- **Discussão:** uma resposta curta por pergunta, na célula de texto da **Resposta**.

Entregáveis

- Um notebook (`.ipynb`) por laboratório da Unidade 2, entregue ao fim da unidade, com a
  respondida nas seções **Resposta**. Nesta unidade não há relatório em PDF.
- O notebook deve **executar do início ao fim sem erros** em um kernel novo (*Restart & Run*
  as saídas exibidas correspondendo a essa execução. Notebooks que não executarem por c
  serão penalizados.
- A entrega é **única por grupo**, com os integrantes identificados no início do notebook.

🧭 Guia de Estudo e Apresentação (Laboratório 9 - n-step SARSA

🟢 O QUE VOCÊ DEVE DETALHAR E DESTACAR (Fale com confiança):

- **A Ponte entre TD(0) e Monte Carlo:** O hBcstep SARSA permite escolher o meio-termo
  perfeito. Com =1$, temos o SARSA convencional (1 passo real + bootstrap); com o \infty
  temos o Método de Monte Carlo (retorno completo sem bootstrap).
- **Atraso nas Atualizações:** O par $ só é atualizado após observar $ recompensas reais r
  futuro, ou seja, $ au = t - n + 1$.
- **Por que o laço continua após o término do episódio?** Porque quando o ambiente ter
  em {end}$, ainda existem -1$ pares pendentes de atualização na memória (buffer). O laç
  mais -1$ iterações sem interagir com o ambiente para atualizar esses pares finais.
- **Desempenho com $ intermediário (=2, 4, 8$):** Acelera muito a propagação do sinal de
  da chegada ao alvo e das quedas no penhasco, reduzindo truncamentos e convergindo r
  mais rápido que =1$.

🔴 O QUE PODE GERAR DÚVIDA / PEGADINHAS DA BANCA (Atenção redobrada):

- **Por que existe a condição dentro do laço?** Porque durante os primeiros -1$ passos (<
  1$), $ au$ é negativo! O agente ainda está apenas acumulando o histórico inicial no buff
  ainda não tem $ passos à frente para calcular o primeiro alvo.
- **O que acontece com o bootstrap nos últimos -1$ passos?** Ele zera! Como o estado $
  terminal, não há ações nem transições após ele. Logo, o termo de estimativa $ desapare
  alvo torna-se exclusivamente a soma descontada das recompensas reais.
- **$ muito grande (=16$) é sempre o melhor?** NÃO! Quando =16$, quase 94% das atuali
  viram Monte Carlo puro. Embora isso elimine o viés das estimativas iniciais de $, introdu

variância gigantesca, pois o retorno passa a depender de 16 ações exploratórias tomada sequência.

0. Implementação

🟢 **O QUE DETALHAR NA TAREFA 0:**

- **Por que o laço continua após o último env.step?** A atualização é defasada em $ pass au = t - n + 1$). Quando o episódio atinge o término em $, os últimos -1$ pares visitados estão na fila aguardando processamento. O laço roda por mais -1$ iterações para esvazi buffer.
- **Com =4$, quais pares são atualizados e o que entra no alvo?** São atualizados 3 pare e $. Seus alvos são somas puras de recompensas reais (sem bootstrap $), pois =T$ é te
  - $ au = T-3 \implies ar{q} = R_{T-2} + \gamma R_{T-1} + \gamma^2 R_T$
  - $ au = T-2 \implies ar{q} = R_{T-1} + \gamma R_T$
  - $ au = T-1 \implies ar{q} = R_T$

🔴 **O QUE PODE GERAR DÚVIDA / PEGADINHA:**

Se o ambiente parou de dar `env.step`, de onde vêm os dados dessas últimas iterações? **Resposta:** Vêm exclusivamente dos buffers de memória `states`, `actions` e `rewards` gravados durante o episódio.

Experimento: implemente `update_policy`, `get_target_value` e `update_Q` de `NStepSarsaAgent` e o laço de `run_episode_and_train`. Execute a seção **Experimento** consiste em:

- treinamento na configuração base;
- avaliação da política aprendida e
- visualização.

Perguntas:

- **(a)** Por que o laço continua depois do último `env.step`? Com $n = 4$, quais pares são atualizados nessas iterações, e o que entra no alvo de cada um?

Resposta

In [ ]:
```
df_base_eval = evaluate_policy(agent)
states, actions, rewards = simulate_trajectory(agent, ENV_ID)
table_base = pd.DataFrame([{
    "Configuração": f"Baseline (n={params.n_steps}, α={params.learning_ra
    "Chegada ao alvo": df_base_eval["chegada ao alvo"],
    "Recompensa média": df_base_eval["recompensa total média"],
    "Duração média": df_base_eval["duração média"],
    "Passos (gulosa)": len(actions),
    "Recompensa (gulosa)": sum(rewards),
```

```
}])
display(table_base)
```

	Configuração	Chegada ao alvo	Recompensa média	Duração média	Passos (gulosa)	Recompe (gulo
0	Baseline (n=4, α=0.01, γ=0.9, ε=0.1, N=5000)	1.0	-21.677	19.202	17	-1

- **(a)**

O laço continua após o último `env.step` porque no n-step Sarsa a atualização do par $(s_\ a_\tau)$ ocorre com um atraso de n passos (com $\tau = t - n + 1$), pois o alvo $\bar q_\tau^ requer a observação das n recompensas subsequentes. Quando o episódio termina no instar (após o último `env.step`), os últimos $n - 1$ pares visitados ainda estão pendentes de atualiz memória. Portanto, o laço precisa continuar executando por mais $n - 1$ iterações, sem interagi ambiente, apenas para processar os alvos desses pares pendentes até que o último par $\tau = seja atualizado.

- Com $n = 4$, ocorrem 3 iterações adicionais pós-episódio para atualizar os pares $(s_{T-3} a_{T-3})$, (s_{T-2}, a_{T-2}) e (s_{T-1}, a_{T-1}).
- O que entra no alvo de cada um:
  - Para $\tau = T - 3$: $\bar q_{T-3}^{(4)} = r_{T-2} + \gamma r_{T-1} + \gamma^2 r_T$
  - Para $\tau = T - 2$: $\bar q_{T-2}^{(4)} = r_{T-1} + \gamma r_T$
  - Para $\tau = T - 1$: $\bar q_{T-1}^{(4)} = r_T$
- Em todas essas iterações finais, como o índice do horizonte atinge o término ($h = T$) e $s terminal, nenhuma estimativa q entra no alvo (*sem bootstrap*): os alvos tornam-se somas das recompensas reais observadas até o encerramento do episódio.

1. Efeito do número de passos (n)

🟢 **O QUE DETALHAR NA TAREFA 1 (n):**

- **Análise dos valores de n:**
  - **=1$:** Mais lento para propagar; alto viés de estimativa; maior número de truncamento início.
  - **\in \{2, 4, 8\}$:** Melhores desempenhos práticos. Propagam as consequências de qu chegada ao alvo muito mais rapidamente, acelerando a convergência.
  - **=16$:** Aproxima-se de Monte Carlo. Praticamente não usa bootstrap na grade de 15- passos.
- **Fração sem bootstrap:** Em um episódio de comprimento médio pprox 17$ passos, os u $ passos dispensam o bootstrap. A fração é $\min(n, L)/L$:
  - =1 pprox 5.9\%$, =2 pprox 11.8\%$, =4 pprox 23.5\%$, =8 pprox 47.1\%$, =16 94.1\%$.
- **Aproximação assintótica:** Quando o \infty$ (ou \ge L$), todas as atualizações usam o r completo do episódio sem bootstrap, tornando-se o **Método de Monte Carlo (MC)** on-po

> 🔴 **O QUE PODE GERAR DÚVIDA / PEGADINHA:**
>
> Por que o n-step SARSA continua aprendendo a rota segura (pelo topo) em vez da rota de 13 passos do Q-Learning? **Resposta:** Porque o n-step SARSA continua sendo *on-policy*! No fina horizonte $, ele ainda faz bootstrap em (S_h, A_h)$ com a ação $ da política $\epsilonhBcgul mantendo a sensibilidade ao risco de queda.

Experimento: treine o n-step Sarsa para $n \in \{1,\ 2,\ 4,\ 8,\ 16\}$, com um ambiente e um a novos para cada valor, fixando:

- $\alpha = 0.01$;
- $\gamma = 0.9$;
- $\epsilon = 0.1$;
- `n_episodes = 5000`;
- os demais parâmetros do baseline.

Crie os parâmetros de cada execução a partir do baseline com `_replace`:

```
params_exp = params._replace(n_steps=8)
```

Mostre uma tabela como a abaixo, com o número de episódios truncados no treinamento e, nas últimas colunas, os resultados de `evaluate_policy` para a política aprendida:

`n_steps`	truncados no treinamento	chegada ao alvo	recompensa total média	duração
1				
2				
4				
8				
16				

Perguntas:

- **(a)** Analise as curvas de treinamento, as trajetórias gulosas e a tabela. Como n afetou o aprendizado?
- **(b)** Use a duração média de cada execução, da tabela, arredondada para o inteiro mais pró
  - em que fração das atualizações de um episódio com essa duração o alvo não usa estir (*bootstrap*)?
  - de que método o n-step Sarsa se aproxima quando n cresce?

Resposta

In [ ]:
```
n_steps_list = [1, 2, 4, 8, 16]
histories_n_steps = {}
results_n_steps = []

for n in n_steps_list:
    label = f"n = {n}"
    params_exp = params._replace(n_steps=n)
```

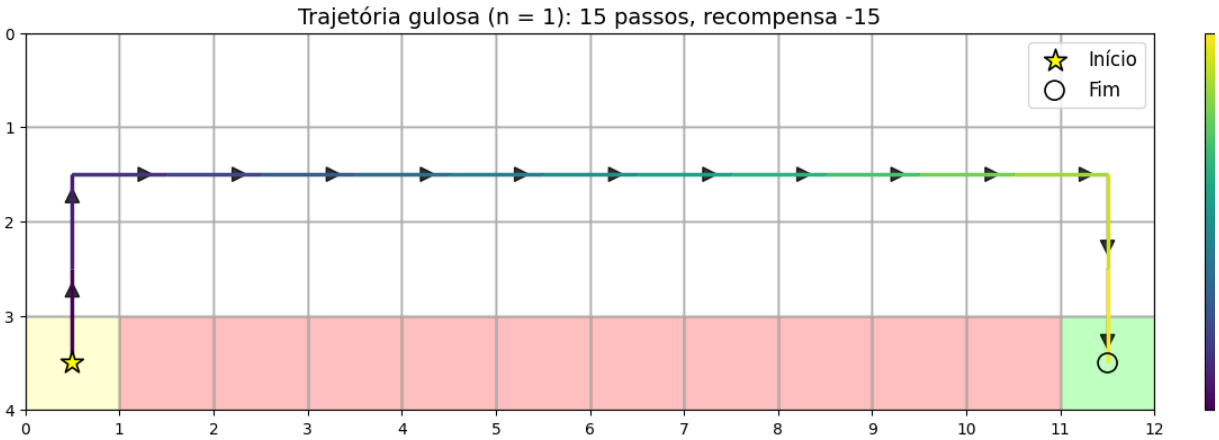
```
    agent_exp = NStepSarsaAgent(params_exp)
    env_exp = make_env()
    history_exp = train_agent(agent_exp, env_exp, params_exp.n_episodes)
    histories_n_steps[label] = history_exp

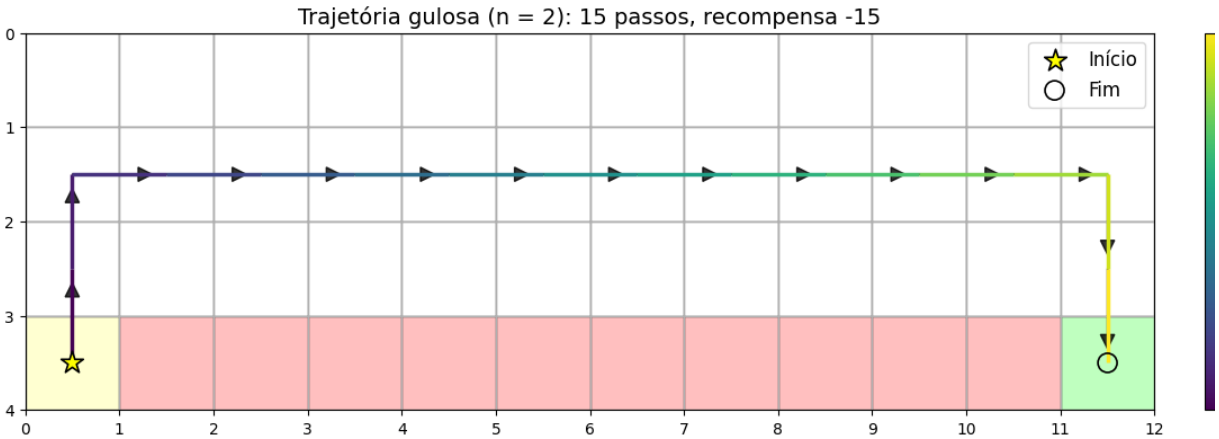
    eval_series = evaluate_policy(agent_exp)
    states, actions, rewards = simulate_trajectory(agent_exp, ENV_ID)
    plot_trajectory(ENV_ID, states, title=f"Trajetória gulosa ({label}):

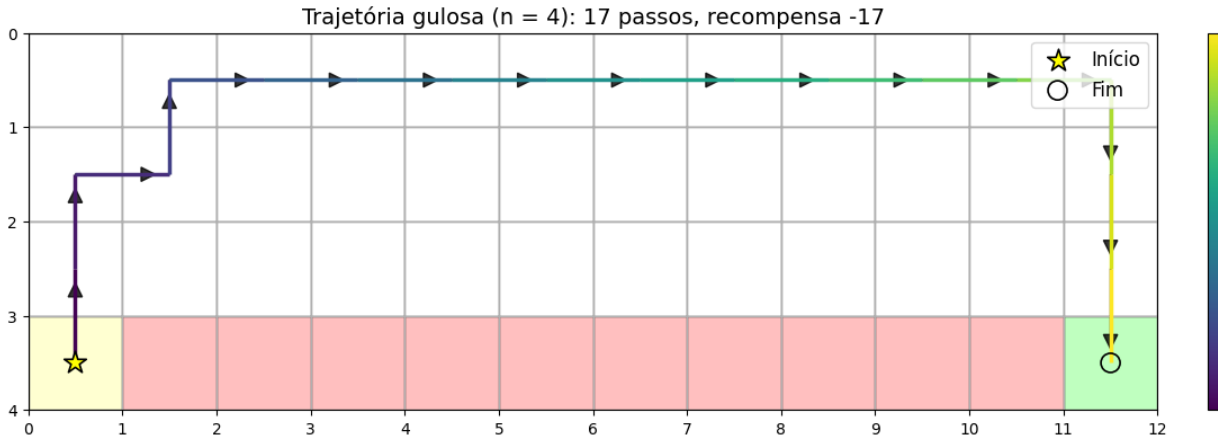
    n_truncated = sum(1 for s in history_exp.successes if s == 0)
    results_n_steps.append({
        "n_steps": n,
        "truncados no treinamento": n_truncated,
        "chegada ao alvo": eval_series["chegada ao alvo"],
        "recompensa total média": eval_series["recompensa total média"],
        "duração média": eval_series["duração média"],
    })

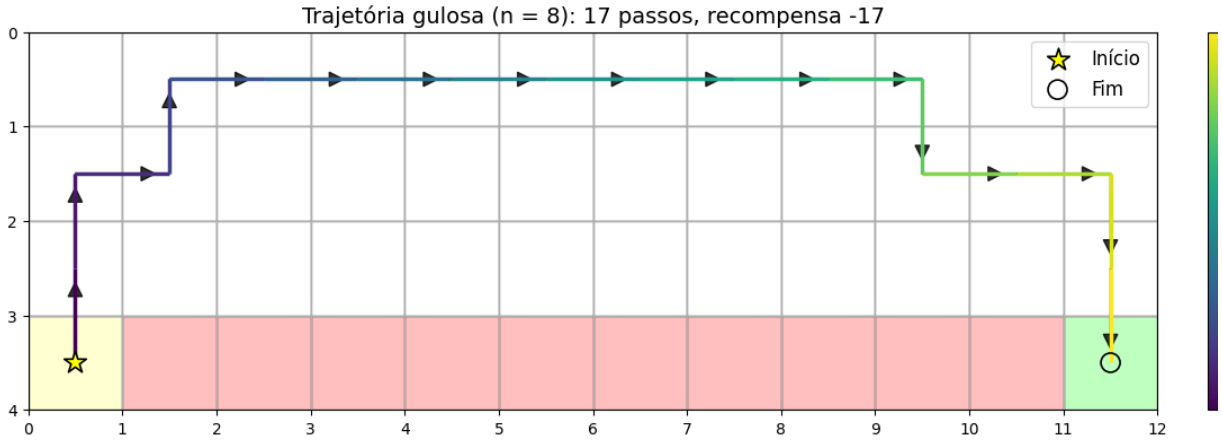
plot_comparison(histories_n_steps, metric="returns", title="Retorno médio
plot_comparison(histories_n_steps, metric="lengths", title="Duração média

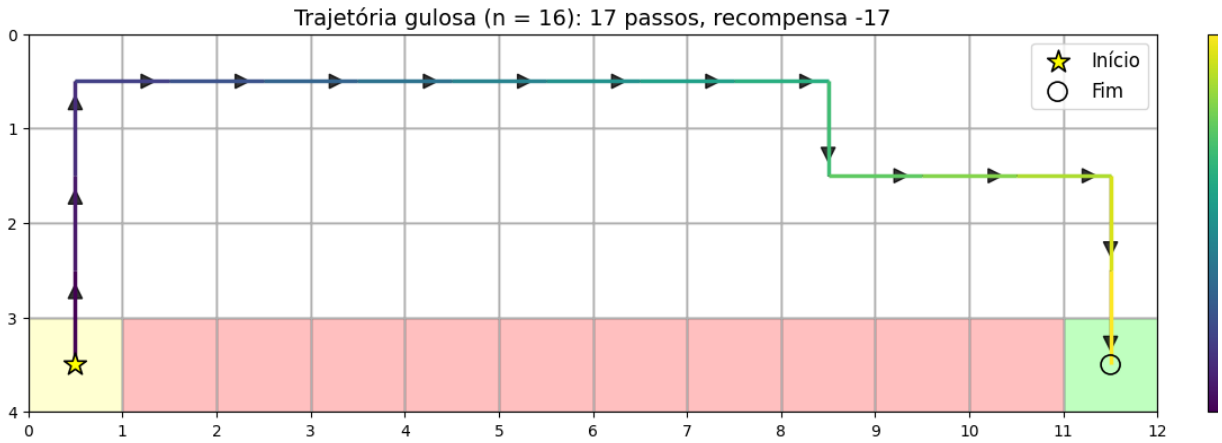
df_table_n = pd.DataFrame(results_n_steps)
display(df_table_n)
```

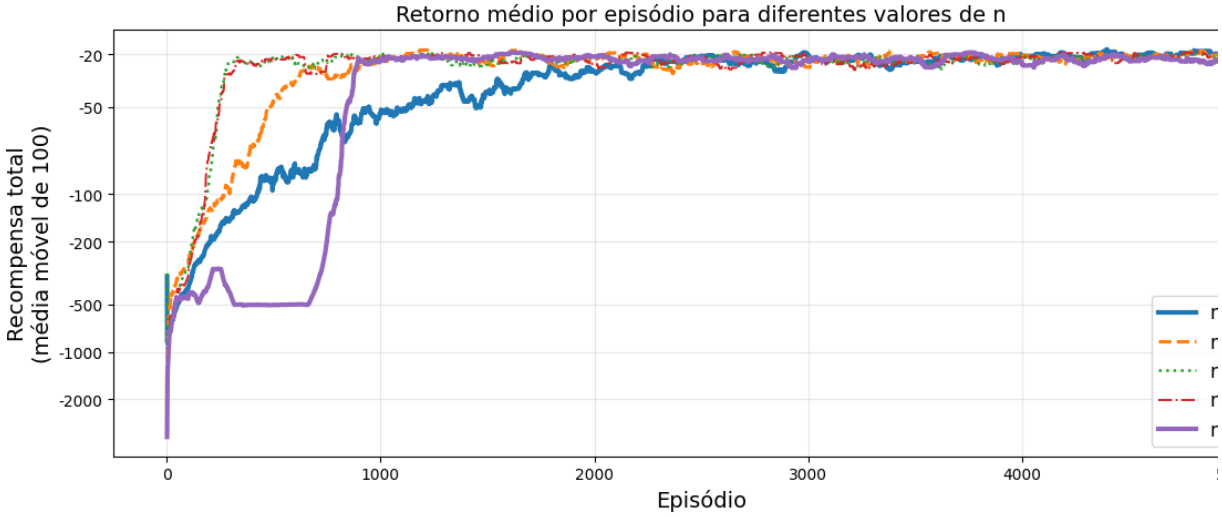


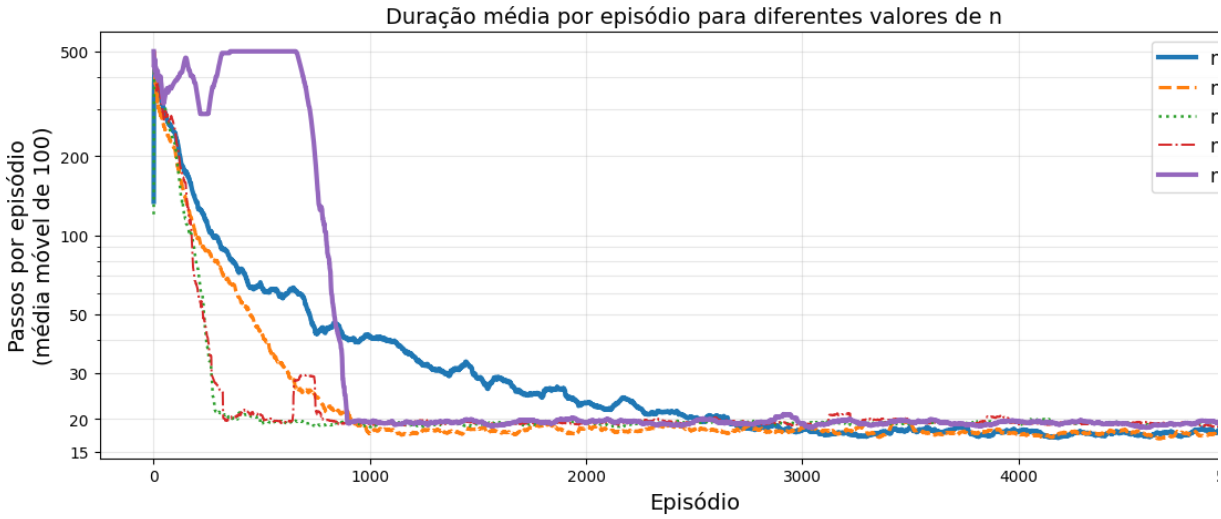



Trajetória gulosa (n = 4): 17 passos, recompensa -17
Início
Fim


Trajetória gulosa (n = 8): 17 passos, recompensa -17
Início
Fim


Trajetória gulosa (n = 16): 17 passos, recompensa -17
Início
Fim


Retorno médio por episódio para diferentes valores de n
Recompensa total
(média móvel de 100)
Episódio



	n_steps	truncados no treinamento	chegada ao alvo	recompensa total média	duração mé
0	1	8	1.0	-26.360	20.3
1	2	8	1.0	-20.956	17.3
2	4	13	1.0	-21.677	19.2
3	8	25	1.0	-22.767	19.2
4	16	560	1.0	-22.055	19.3

- **(a)** Efeito de n no aprendizado:
  - **$n = 1$:** O aprendizado é mais lento para propagar o sinal de reforço da chegada ao a origem, pois cada atualização depende do bootstrap imediato de uma única transição. Apresenta o maior número de episódios truncados no início do treinamento.
  - **$n \in \{2, 4, 8\}$:** O desempenho melhora substancialmente. Ao acumular múltiplas recompensas reais observadas antes de recorrer à estimativa de bootstrap, o agente a penalidade das quedas e o caminho até o alvo muito mais rapidamente, acelerando a convergência e reduzindo os truncamentos. Todos convergem de forma estável para a segura do Sarsa (afastada do penhasco).
  - **$n = 16$:** Como a duração típica de um episódio no CliffWalking é em torno de 15 a 17 $n = 16$ faz com que quase todas as atualizações usem o retorno completo sem boots Isso elimina o viés das estimativas iniciais de Q, mas introduz maior variância devido estocasticidade das ações exploratórias tomadas ao longo de muitos passos.
- **(b)** Análise da fração sem bootstrap e aproximação metodológica:
  - **Fração das atualizações sem bootstrap:** Em um episódio com duração média de L ($T = L$), realizam-se L atualizações. O alvo dispensa o bootstrap nos últimos n p (onde $\tau + n \ge L$). Assim, a fração de atualizações sem bootstrap é dada por $\m L$. Tomando a duração média típica arredondada de $L \approx 17$ passos:
    - Para $n = 1$: $1 / 17 \approx 5.9\%$
    - Para $n = 2$: $2 / 17 \approx 11.8\%$
    - Para $n = 4$: $4 / 17 \approx 23.5\%$
    - Para $n = 8$: $8 / 17 \approx 47.1\%$
    - Para $n = 16$: $16 / 17 \approx 94.1\%$

- **Aproximação quando n cresce:** À medida que $n \to \infty$ (ou quando $n \ge L$) passa a somar todas as recompensas reais até o fim do episódio sem recorrer a nenhu estimativa q. Portanto, o n-step Sarsa aproxima-se do **Método de Monte Carlo (M** policy.